Unidad 3:

LAB-01 Bitácoras y auditoría del dato en PostgreSQL.

PARTE B:

Auditoría del dato con disparador.

Actividad 11:

Crear la función y el disparador.

Alumno: Cesar Alberto Lopez Zavala. Fecha: 24/10/20.

Preguntas:

¿Cómo te ayudó el trigger a resolver el problema que la bitácora del servidor no pudo resolver en la Actividad 7?

El trigger nos permitió guardar el valor anterior exacto en formato JSON. La bitácora del servidor solo registraba que se había ejecutado un comando, pero sin los datos originales, era humanamente imposible reconstruir y recuperar el registro perdido.

En el script de recuperación, ¿por qué es necesario hacer un "casteo" (ej. ::BIGINT o ::INTEGER) al extraer los datos del JSON?

Porque al extraer valores de un objeto JSON en PostgreSQL usando el operador ->>, el motor devuelve todo como texto plano. Si la tabla clientes espera un número o una fecha, debemos convertirlo explícitamente para que los tipos de datos coincidan y la inserción no falle.

Si el usuario malintencionado borra la tabla de clientes por completo usando TRUNCATE o DROP TABLE, ¿el trigger guardará los datos?

No. Los triggers de tipo FOR EACH ROW operan fila por fila y no se disparan con operaciones DDL masivas como TRUNCATE o DROP. Para auditar esos comandos destructivos dependeríamos de la bitácora del servidor, pero los datos se perderían instantáneamente y tendríamos que recurrir a un respaldo tradicional para recuperarlos.

Evidencia de ejecución:

```
1  -- Unidad 3. LAB-01 Bitácoras y auditoría del dato en PostgreSQL.
2  -- Actividad 11 – Crear la función y el disparador.
3  -- PARTE B. Auditoría del dato con disparador.
4
5  SELECT id_auditoria, operacion, usuario_db, fecha_evento, datos_anteriores
6  FROM auditoria.historial_clientes
7  ORDER BY id_auditoria DESC LIMIT 1;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	id_auditoria [PK] bigint	operacion character varying (10)	usuario_db character varying (50)	fecha_evento timestamp with time zone	datos_anteriores jsonb
1	2	DELETE	postgres	2026-11-07 23:07:44.525204-...	{"id": 999, "rfc": "XAXX010101000", "edad": 30, "ciudad": "CDMX", "correo": "prueba@test.com", "nombre": "Cliente

```
9   INSERT INTO public.clientes (id, nombre, corre
10  SELECT
11      (datos_anteriores->>'id')::BIGINT,
12      datos_anteriores->>'nombre',
13      datos_anteriores->>'correo',
```

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 49 msec.
```